Milestone 4 Alpha build and playtest packet

I will first construct the <theory-of-the-program>, then generate <program text> only after the theory is explicit.

OPERATIONAL_DIAGNOSTIC

Substantives_Demolished Replace vague uses of usability, feedback and improvement with the actions and records below.

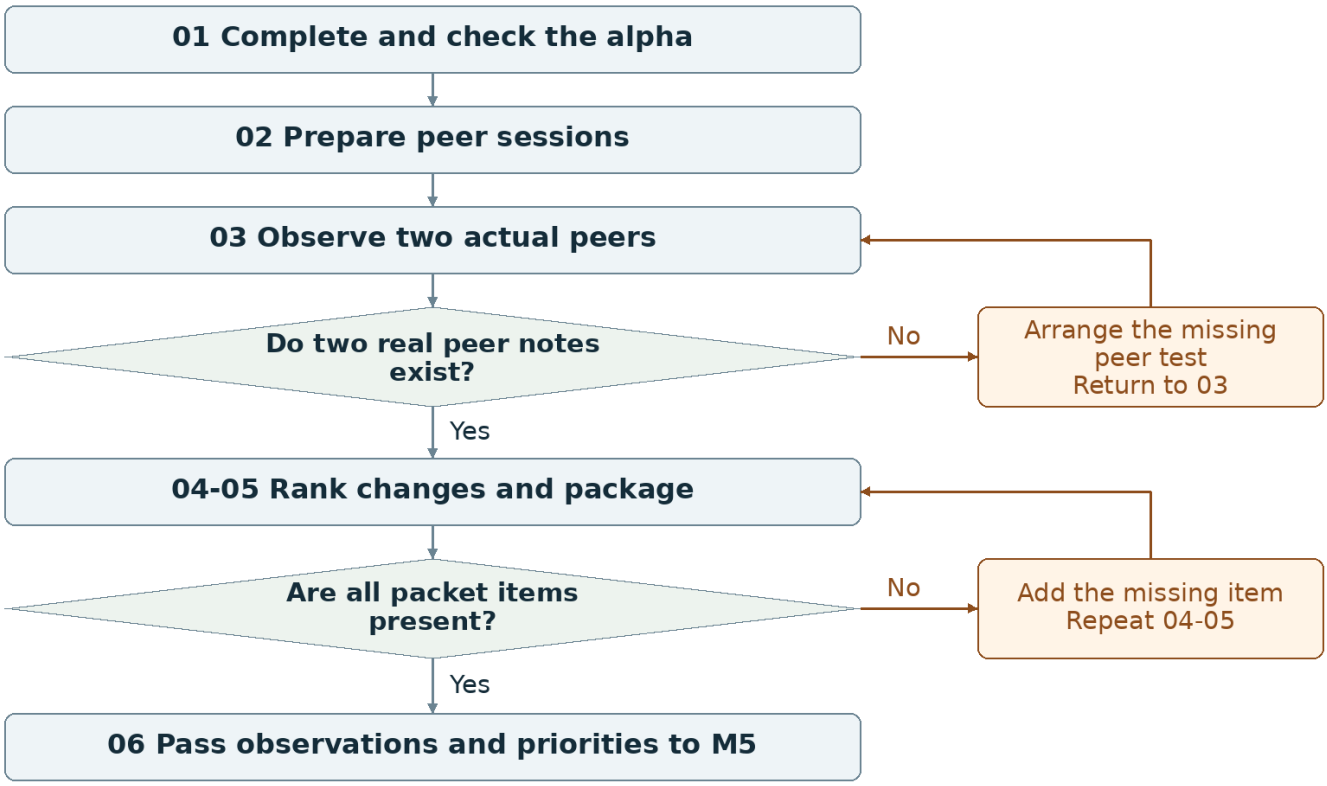
Occult_States_Removed Replace users were confused with the control they tried, the display they saw, the point they stopped and any instruction you gave.

The_Language_Game Record peer actions first; connect each proposed change to an observation or a labeled group refinement.

CONTROL_SURFACE_DRAFT

Complete the route from beginning to end, observe two actual peers using it, and rank three next changes from recorded evidence.

Start with Open the M3 prototype, current rules, launch record, missing-work list and defects. Obtain the assigned feedback form when available and arrange actual peer sessions.



Use the numbered procedure; repeat checks only where the repair affects their result.

Required package Submit one ZIP containing the complete alpha build and playtest packet.

Assign maker _____ and checker _____. Swap roles at the next task. Leave checks pending until performed.

Milestone 4 Write the rules

1 <Initial Interpretation>

Write the task each peer will attempt and the starting condition. Record the intended ending and what counts as an intervention. Use the session to observe actions, stops and outcomes without adding explanations that the release does not provide.

2 <Theory Skeleton>

Record	What to write
<Entities>	Track the alpha build, peer, task, attempted input, displayed response, observation and proposed change.
[Operations]	Launch, give initial instructions, observe, record intervention, collect a note and rank revisions.
<States>	Keep the complete program route and its endings. Record the exact state in which each peer stops or changes approach.
<Conditions>	For each recorded peer action, identify the input guard and ending condition checked by the program. Keep session and peer-note checks in the procedure.
<Invariants>	Name the adopted runtime rules that must remain true within their scope. Check their affected values during alpha completion and peer input sequences.
<Constraints>	Supply two peer notes and three next changes. Preserve the tested build identifier and the source of each feedback statement.

Use conditions to decide whether an action or transition is allowed. State the scope of each invariant and how to check it. Keep submission requirements separate from runtime rules.

3 <Assumption Ledger>

Status	Action
<safe>	Enter an unverified premise only with its basis and limited consequence if wrong. Keep prior observations labeled with the build and sequence checked.
<uncertain>	Mark whether a new peer can use the instructions and reach an ending as untested until a session occurs.
<requires-user-decision>	Have the group select the three revisions. Keep a peer suggestion separate from an adopted requirement.

For each suspected problem, record {observed event or prediction, affected build/state, next check, owner}. Do not infer a peer's motive from a pause or wrong input.

Record the next open item Claim _____ Status _____
Owner _____ Evidence/check _____ Next action _____

Write <entity> for a person, object, value or state; [operation] for an action; and {record} for related fields. Record planned, observed, failed or not tested beside each claim. Do not mark a check passed until a person performs it and records the result.

Milestone 4 Run the procedure

4 <Operational Description>

Read each action aloud before doing it. The maker performs the action; the checker records the named result. Use the return instruction when a check fails.

Step	Perform and save	Check and return
01 Finish the alpha	Complete the required beginning, middle and ending. Keep existing controls and feedback usable. Resolve prototype defects that prevent completion.	Run a full route internally and record the result before giving that version to a peer.
02 Prepare the sessions	Select the exact alpha and a short task instruction. Arrange two peers; provide the assigned form when available. Give both sessions the same initial conditions when comparing behavior.	Keep the form's actual fields. Do not fabricate its contents or claim a session that has not occurred.
03 Observe two peers	One partner watches actions; the other records the state, input, response, stop point and any intervention. Avoid coaching the route. Ask a bounded follow-up about an observed difficulty.	Save two genuine notes. Label quotations, paraphrases and group interpretations separately. AI roleplay and self-review do not replace peers.
04 Rank three changes	For each priority, record the observed problem, proposed change, owner and repeatable check. Rank completion blockers before cosmetic work. Label a group refinement when it was not a peer complaint.	If fewer than three issues were observed, use an honest group refinement for a remaining slot; do not invent a third complaint.
05 Package the packet	Include the alpha, two peer notes, top-three revision plan, updated screenshot set, progress note and AI statement in one ZIP. Label the build shown in each record.	Extract the ZIP and open the build and every note. Repair a missing file or mismatched build reference.
06 Hand off	Give M5 the alpha, the raw observations, three ranked changes and checks. Record which peer tasks completed and which stopped.	Leave the stage pending if the build cannot finish or a real peer note is missing.

5 <Failure Description>

Observed problem	Required response
Peer cannot reach an outcome	Record the stop and intervention, repair the alpha, and rerun the affected route; preserve the original note.
Only one real peer note exists	Arrange and run the missing session in step 03. Do not create substitute testimony.
Revision says make it clearer	Name the label, control or response to change and the exact peer action to repeat in step 04.

Use the map A no branch returns to the named task. Repair the recorded problem and repeat that check; do not mark the stage passed because the planned time window ended.

Milestone 4 Test changes and produce the work

6 <Change Test>

Ask: "What would change if the requirements changed?" The maker writes the expected result first. The checker performs or traces the change and records the result. Swap roles for the second scenario.

Modification	Run this check
Use an unplanned legal sequence	Select a valid sequence the designers did not rehearse. Predict the ending or permitted continuation, then run it. Record unreachable states, hidden ordering requirements or contradictory results.
Remove spoken guidance	Give a peer only the instructions shipped with the alpha. Record every stop and any intervention. Repeat the affected task after the relevant written instruction changes.

Save {version or design record, starting condition, input/change, expected result, observed result, pass/fail/not tested, next action}. Use paper traces in M1-M2; report executed checks only when they occurred.

7 <Implementation Plan>

One partner completes a missing route element while the other prepares the task and record sheet; swap review before testing. During sessions assign one observer and one recorder. Preserve the tested version so both partners can compare earlier and revised runs during M5.

8 <Program Text>

Make only the implementation changes needed to complete the route and remove blocking defects. Write the expected behavior before each change, record the actual result, and keep assisted-code comments current. Do not manufacture peer observations to justify a patch.

9 <Theory-Code Mapping>

Element	Record the connection
<types>	Locate data involved in each observed problem.
<functions>	Name the input, update or display function related to each revision.
<classes>	Record affected responsibilities only where classes exist.
<tests>	Turn the observed failing action sequence into a repeatable check.
<comments>	Record the rule reason or prior defect when it helps a later edit.
<configuration>	Mark adjustable labels, timings or limits; preserve tested values.

Use actual file and function names when code exists. Label planned counterparts proposed; write not applicable when a structure is absent.

Carry forward the course rules

Keep the project connected to WarGames. Build in Processing using p5.js mode; accept interactive input; include at least three distinct screens or states; provide a start and reachable outcome; credit external material; organize and comment the code.

Record actual AI use at every milestone, including use of this manual if adopted. Use AI for bounded brainstorming, explanation, small examples you study and adapt, specific debugging or review. Author the project as a group; do not request an entire generated project. Identify assisted code and your changes in comments.

Milestone 4 Submit and hand off

10 <Residual Human Theory>

Record why the group ranked the three changes in this order and which suggestions it did not adopt. Preserve raw notes beside those decisions. Each partner traces one observation to a proposed change and its pass/fail check.

Check the required submission

- One ZIP containing an alpha playable from beginning to end.
- Two actual peer feedback notes; use the assigned form when available.
- A revision plan listing the top three next changes.
- An updated screenshot set and short progress note.
- AI Use Statement and current credit/reference records for included material.

Suggested ZIP contents: project folder, peer note 1, peer note 2, revision plan, screenshots, progress note and AI record. The assignment sets no PDF minimum for this stage.

Completion check The checker completes the alpha route, opens two genuine peer notes and finds three ranked changes with stated checks. Every required item opens from the extracted ZIP.

Progress note Name the current version; state what was completed and checked, what is missing, and the next action with its owner. Add the screenshot/capture supporting the current status.

Complete the AI Use Statement

Field	Enter the actual record
Tool and dates	Name the tool, or record no AI used only if accurate. Enter actual dates of use.
Request and affected work	Summarize the request. Name the text, image, planning record or code file affected.
Kept changed rejected	Record the output retained, edits or rewrites made, and material discarded.
Group confirmation	Recommended check: each partner traces actual submitted code; for planning-only work, trace the proposed interaction. Identify image sources, retained edits and displayed information. Then complete the required group confirmation covering code and images; leave it pending until both partners can confirm it.

If this AI-produced manual is adopted, record ChatGPT, October 21 2026, milestone planning and operational rewrite, and your actual kept/changed/rejected choices. Do not copy a completed student confirmation from this manual.

Handoff Pass observed behavior and three executable revision tasks to M5. Keep missing peer evidence visible; do not treat a completed internal test as a second peer session.

Record: maker _____ checker _____ version _____
Checks still pending _____ submitted file(s) _____ upload status _____

Requirements source: supplied WarGames project brief and milestone instructions. Treat roles, file names, internal checks and relative-week allocations as recommended working procedures. Use corrected deadlines from the course; none are asserted here.